

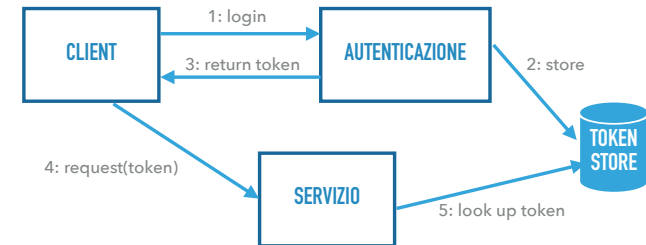
Accesso Sicuro ai Servizi

- ▶ In un sistema distribuito le richieste ai servizi (lato server) devono essere vagliate: solo gli utenti autenticati e autorizzati posso usare i servizi
- ▶ Bisogna far sì che ciascuna richiesta sia verificata e solo se autorizzata sia eseguita, tuttavia è necessario evitare di chiedere continuamente le credenziali agli utenti
- ▶ Inoltre bisogna far sì che la prova di identità e la verifica delle credenziali sia gestita in un solo server



Prof. Tramontana - October 2025

Autenticazione Basata su Token



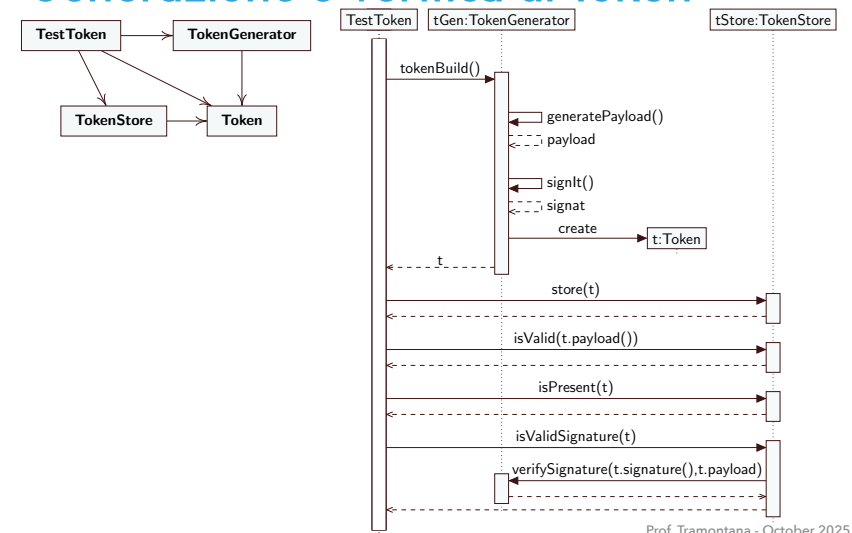
Prof. Tramontana - October 2025

Token

- ▶ Una volta autenticato l'utente, gli si fornisce un token che userà per le sue successive richieste ai servizi
- ▶ Un token per l'accesso è una credenziale che può essere usata per accedere a servizi
- ▶ Un token può essere una stringa (opaque, ovvero incomprensibile), per es. `kPoPMRYrCEoYO6s5` oppure un oggetto, per es. un oggetto JSON, detto JSON Web Token (JWT)
- ▶ Lo scopo del token è informare che chi lo possiede è stato autorizzato ad accedere al servizio
- ▶ Il token tipicamente indica: l'utente identificato, chi lo ha emesso, il timestamp al momento dell'emissione, la durata, payload, una signature

Prof. Tramontana - October 2025

Generazione e Verifica di Token



Prof. Tramontana - October 2025

JSON Web Token (JWT)

- ▶ I JWT (pronuncia jot) sono un formato standard per token di sicurezza auto-contenuti
- ▶ Un JWT consiste di un set di diritti (claim) per un utente rappresentati tramite un oggetto JSON, insieme a un header (intestazione) che descrive il formato del token
- ▶ I JWT sono protetti crittograficamente e possono essere criptati
- ▶ L'oggetto JSON può contenere gli attributi della sessione; viene creato dal server, mandato al client, e successivamente quando il server lo riceve recupera gli attributi della sessione

Prof. Tramontana - October 2025

Firma Crittografica del Token

- ▶ Per evitare che chiunque crei il token o modifichi gli attributi del token, si aggiunge un campo calcolato in base a una chiave segreta

Prof. Tramontana - October 2025

Claim Standard per JWT

- ▶ Alcuni claim standard (RFC 7519) sono
 - ▶ Issuer: `iss String`, chi ha creato il token
 - ▶ Subject: `sub String`, l'identificativo univoco dell'utente
 - ▶ Audience: `aud String` o `array`, riceventi previsti per il token
 - ▶ Not before: `nbf number`, token non valido se usato prima di questo istante
 - ▶ Issued at: `iat number`, quando il token è stato creato
 - ▶ Expiration time: `exp number`, fine della validità del token
 - ▶ Id: `jti String`, identificativo del token

Prof. Tramontana - October 2025

Libreria Auth0

- ▶ Una libreria molto usata per il supporto alla generazione e verifica dei token è auth0
- ▶ Aggiungere al file di progetto (POM) maven la dipendenza

```
<dependency>
  <groupId>com.auth0</groupId>
  <artifactId>java-jwt</artifactId>
  <version>4.5.0</version>
</dependency>
```

Prof. Tramontana - October 2025

Creazione del Token

```
String token = JWT.create()
    .withIssuer("auth0-my-demo")
    .withSubject("my-user-123")
    .withIssuedAt(new Date())
    .withExpiresAt(new Date(System.currentTimeMillis() + 3600_000))
    .withJWTId("token-xyz-123")
    .sign(algorithm);
```

- ▶ Il token creato contiene l'id di chi lo ha prodotto, l'id dell'utente, la data di creazione, la data di scadenza, l'id del token, e quindi la firma crittografica
- ▶ Il metodo sign() prende una istanza che rappresenta l'algoritmo di crittografia per la firma e per la verifica

```
Algorithm algorithm = Algorithm.RSA256(publicKey, privateKey);
```

Verifica del Token

```
JWTVerifier verifier = JWT.require(algorithm)
    .withIssuer("auth0-my-demo")
    .build();

DecodedJWT decoded = verifier.verify(token);

System.out.println("Subject: " + decoded.getSubject());
```

- ▶ Il metodo verify() lancia una eccezione se il token non è valido